SUBJECT: WINDOWS AND LINUX INTERNALS

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

WINDOWS MEMORY BEHAVIOR AND PAGE-FAULT ANALYZER

Presented by

Akash S – RA2312703010006

Homesh Krishnan U – RA2312703010022

Mohammed Yasir – RA2312703010032

Department of Networking And Communications

Academic Year 2025–2026

FOUNDATIONS AND PROBLEM

PART I

“Not all virtual pages need to be RAM-resident at once.”

INTRODUCTION

Memory Management in Modern Operating Systems

PHYSICAL VS VIRTUAL MEMORY

- Every process gets its own isolated virtual address space
- Physical RAM is shared across ALL processes by the OS
- Virtual pages are mapped to physical frames on demand
- Not all virtual pages need to be RAM-resident at once
- Windows demand-paging: pages brought in only on first access
  - This first-access event is the fundamental page fault

THE PAGE FAULT MECHANISM

- CPU accesses address not mapped → hardware exception raised
- Windows Memory Manager intercepts and resolves the fault
- Resolution may be disk-free (soft) or need disk I/O (hard)
  - Soft: zero-fill, standby list, copy-on-write — no disk
  - Hard: page read from pagefile — actual disk I/O occurs
- Windows PageFaultCount counts ALL types together

WHY THIS MATTERS

- Memory is the #1 performance bottleneck in most workloads
- Different access patterns cause measurably different fault activity
- Task Manager shows totals — not behavior over time
- Controlled experiments reveal OS memory decisions
- Critical for optimizing real-world software
- Pagefile activity is invisible to most developers

Problem statement & objectives

RESEARCH QUESTION

How do memory allocation and access patterns affect page-fault activity on Windows?

THE PROBLEM

Standard monitoring tools (Task Manager, Resource Monitor) report instantaneous totals but do not reveal how different memory allocation strategies influence working-set growth, page-fault accumulation, or physical RAM pressure over time. No built-in tool runs controlled workloads to isolate these effects.

01

Experiment

Run 5 controlled workloads: Allocation · Sequential · Repeated · Random · Release

02

Measure

Per-interval capture of Working Set · Private Bytes · PageFaultCount · Available RAM

03

Analyse

Compute fault Δ and fault rate. Plotly charts: trend, bar, overlay. Side-by-side comparison

04

Persist & Export

SQLite (WAL mode) for all data. CSV export: summaries + samples. Delete history with confirmation

Operating system memory concepts

Key Windows internals underpinning the application's measurements

Working Set

Physical pages currently mapped into the process. Reported by GetProcessMemoryInfo as WorkingSetSize. Evicted pages leave the working set; re-accessing them causes a fault.

Private Bytes

Virtual memory committed exclusively to this process (PagefileUsage). Backed by pagefile or physical RAM. A process can have large private bytes but a small working set.

Virtual Size

Total user-mode virtual address space reserved/committed. Can exceed RAM; only accessed pages consume physical memory via demand-paging.

Page Fault Count

Cumulative DWORD counter (PageFaultCount). Incremented on every page access not currently in the working set. Counts soft faults AND hard faults — cannot distinguish them.

PAGE FAULT RESOLUTION TYPES

Zero-demand — No disk

New private page; OS zero-fills without reading disk

Soft fault — No disk

Page on standby list; re-mapped into working set

Copy-on-write — No disk

Shared page copied to private — no disk read

Hard fault — Disk I/O

Page read from pagefile or mapped file — disk I/O

PageFaultCount counts ALL types — cannot separate hard from soft faults

SYSTEM DESIGN

PART II

“Real modules, real data flows.” Derived from the actual source code.

Proposed system & features

A controlled-experiment platform — goes beyond passive monitoring

The application deliberately induces measurable memory-management activity and records the Windows OS response at every configurable sampling interval (0.5 s – 60 s).

01 System Dashboard

RAM · CPU · pagefile gauges
Virtual memory · kernel counters
Rolling trend charts (120 pts)
GlobalMemoryStatusEx + GetPerformanceInfo

02 Process Inspector

Search & select any running PID
Working set · private bytes
PageFaultCount · pool usage
GetProcessMemoryInfo (PMCEX)

03 Memory Experiments

5 workloads (A–E) in subprocess
UI stays responsive during run
Start · Stop · Reset · Save
Alloc 4–256 MiB · 3–300 s

04 PF Analysis

Initial / final / delta / rate
Dual-axis: working set + faults
Per-interval Δ bar chart
Counter limitations panel

05 Comparison

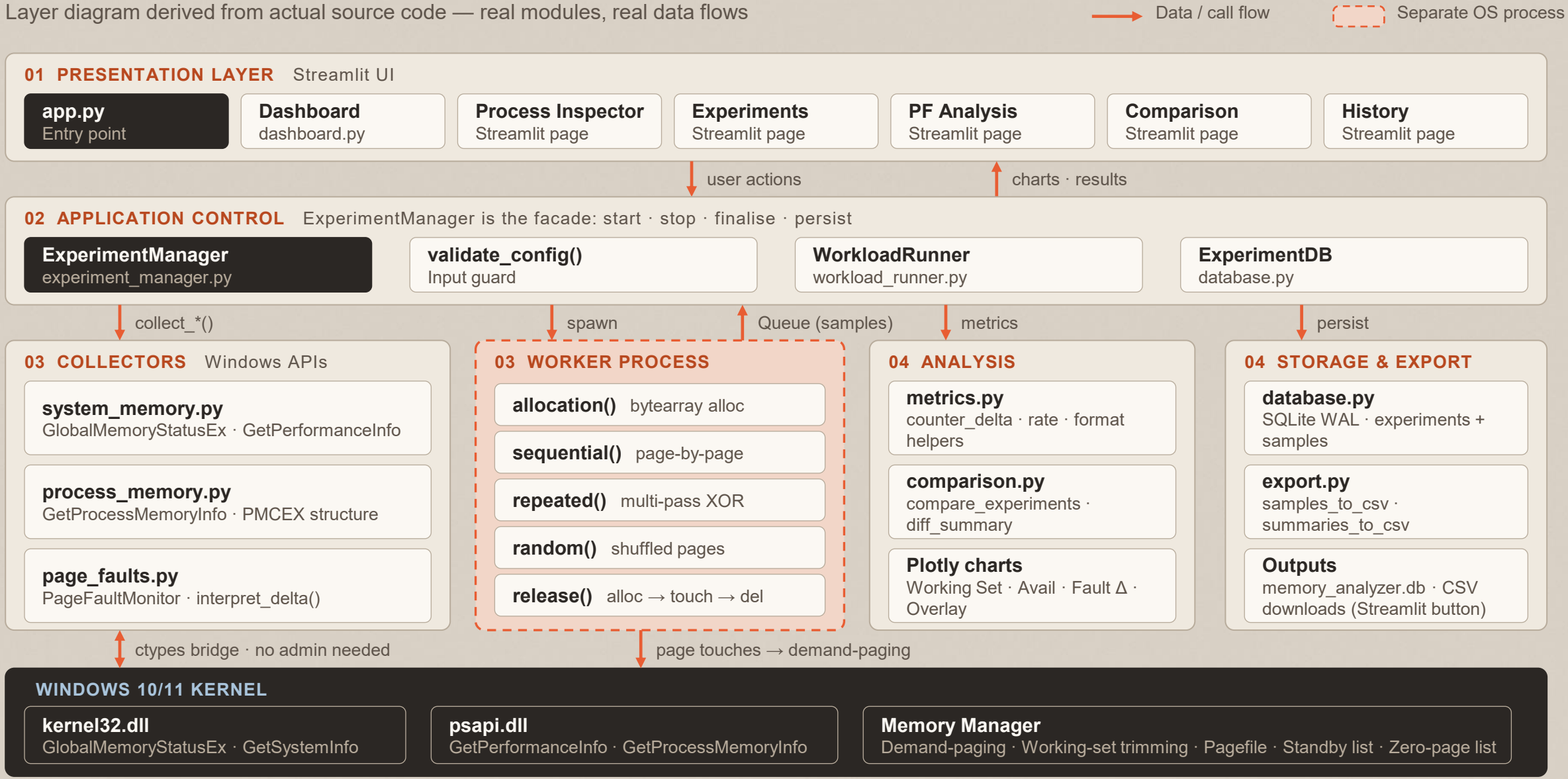
Multi-select 2+ experiments
Side-by-side table + bar charts
Working-set overlay plots
Page-fault Δ overlay

06 History & Export

SQLite: metadata + samples
Export summaries CSV
Export raw samples CSV
Delete with confirmation

System architecture

Layer diagram derived from actual source code — real modules, real data flows



Application workflow

Step-by-step flow: launch → data collection → experiment → analysis → export

STEP 01

Launch

streamlit run app.py
localhost:8501
ExperimentManager initialised

STEP 02

Dashboard

collect_system_memory()
GlobalMemoryStatusEx
GetPerformanceInfo
Gauges + charts

STEP 03

Inspect PID

User selects PID
collect_process_memory()
GetProcessMemoryInfo
Working set shown

STEP 04

Config

Pick workload A–E
Set alloc · duration · interval · seed
validate_config()

STEP 05

Run Workload

WorkloadRunner.start()
Spawns subprocess
Queue drainer thread

STEP 06

Monitor

samples → session_state
_snapshot() per tick
page_fault_count
working_set_mb · Plotly rerun

STEP 07

Finalise

finalise() called
Summary → SQLite
Fault Δ computed
Interpretation shown

STEP 08

Export

Comparison page
Overlay charts
CSV download
st.download_button()

SUBPROCESS ARCHITECTURE · STEPS 5–6

WorkloadRunner spawns multiprocessing.Process(daemon=True)

Workload function runs independently

Samples emitted via multiprocessing.Queue

Background reader thread drains queue into session_state

stop() calls process.terminate() + join(timeout=5)

Cancelled experiments marked in SQLite

Modules & technologies

Complete module inventory with responsibilities and technology mapping

MODULE / FILE	RESPONSIBILITY	KEY API / LIBRARY
collectors/system_memory.py	Collects RAM, pagefile, virtual memory, CPU, process/thread counts	ctypes · GlobalMemoryStatusEx · GetPerformanceInfo
collectors/process_memory.py	Per-process working set, private bytes, PageFaultCount, pool quota	ctypes · GetProcessMemoryInfo · PROCESS_MEMORY_COUNTERS_EX
collectors/page_faults.py	PageFaultMonitor class, PageFaultSeries, interpret_page_fault_delta()	Python dataclasses · process_memory
experiments/memory_workloads.py	5 workload functions running in worker process, emitting Queue samples	multiprocessing.Queue · bytearray · ctypes
experiments/workload_runner.py	WorkloadRunner: subprocess lifecycle, queue drain thread, cancellation	multiprocessing.Process · threading.Thread · threading.Event
experiments/experiment_manager.py	Facade: start/stop/finalise/persist, page-fault interpretation	WorkloadRunner · ExperimentDB · interpret_page_fault_delta()
analysis/metrics.py	Unit conversion, counter_delta, counter_rate, counter catalogue	Pure Python (no OS deps) — fully unit-testable
analysis/comparison.py	compare_experiments(), diff_summary(), compatibility warnings	Pure Python
storage/database.py	SQLite schema, experiment CRUD, sample insert, cascade delete	sqlite3 WAL mode · json · contextmanager
storage/export.py	samples_to_csv(), summaries_to_csv() for download buttons	csv.DictWriter · io.StringIO
ui/*.py (7 pages)	Streamlit pages: dashboard · inspector · experiments · PF · compare · history · about	Streamlit · Plotly · pandas
config.py	Global constants: alloc limits, workload names, colours, DB path	os.path

TECHNOLOGY STACK

Python 3.12	**Streamlit ≥ 1.32**	**Plotly ≥ 5.19**	**psutil ≥ 5.9**	**ctypes (stdlib)**	**sqlite3 (stdlib)**	**csv (stdlib)**	**pandas ≥ 2.0**	**pytest ≥ 8.0**
Language	Web UI	Interactive charts	Portable metrics	Windows API bridge	Persistence	Data export	UI DataFrames	97 tests

EXPERIMENTS AND RESULTS

PART III

“97 / 97 automated tests actually run and passed — not claimed.”

Memory experiments & analysis

5 controlled workloads — implementation, mechanisms, and expected observations

		METHOD	WHY	EXPECTED OBSERVATION
A	**Allocation**	bytearray(N) — commits virtual memory without deliberately accessing every page	Zero-demand faults from OS initialising allocator metadata; virtual size > working set	Fault Δ small; working set grows slowly; large gap between virtual size and WS
B	**Sequential**	Writes one byte to each page in address order, stride = page_size (4096 B)	First touch of each page triggers a demand-zero fault; working set rises linearly	Expected Δ ≈ alloc_bytes / page_size (64 MiB → ~16 384 faults); largest Δ of all workloads
C	**Repeated**	Touches all pages once (first pass), then XOR-reads the buffer multiple times	Pass 1 generates max faults; subsequent passes: pages already resident → near-zero extra faults	Fault rate drops sharply after pass 1; working set stabilises at full allocation size
D	**Random**	Shuffled page index list (configurable seed) — accesses pages in pseudorandom order	Same total pages touched as sequential → similar fault Δ on uncongested RAM machine	On low-RAM systems TLB/cache miss difference observable; seed makes run reproducible
E	**Release**	Allocate → touch all pages → hold → del buf → monitor post-release for remaining duration	Working set drops after release; system RAM may stay elevated (standby list effect)	Demonstrates that freeing memory ≠ immediate return to OS physical pool

Each experiment records per interval: page_fault_count (cumulative) · working_set_mb · virtual_mem_mb · available_ram_mb · system_memory_pct — stored in SQLite, visualised in Plotly

GUI & live demonstration

7 Streamlit pages at http://localhost:8501 — runs without admin privileges

SEVEN PAGES

Dashboard
RAM/CPU/Pagefile gauges · virtual memory metrics · kernel counters · rolling trend charts

Process Inspector
Search by name · bar chart of WS/private bytes · PageFaultCount · counter definition panel

Memory Experiments
Sidebar config (workload, size, duration) · Start/Stop/Reset · live Plotly charts during run

PF Analysis
Fault Δ/rate metrics · dual-axis (WS + cumulative PF) · per-interval Δ bar · limitations panel

Comparison
Multi-select experiments · side-by-side table · bar charts · WS overlay · PF-Δ overlay

History
Summary table (pandas) · individual detail view · CSV download · delete with confirmation

About & Metrics
Windows concepts · page fault type table · full counter catalogue · tech stack table

LIVE DEMO SEQUENCE (~6 MINUTES)

01	Open Dashboard → observe RAM %, pagefile gauge, trend chart
02	Process Inspector → select python.exe → note PageFaultCount value
03	Experiments → B Sequential, 64 MiB, 12 s → click Start
04	Watch live working-set chart rise as pages are faulted in
05	After completion → PF Analysis → review fault Δ, rate, interpretation
06	Experiments → D Random, same settings → Start and complete
07	Comparison → select both → overlay charts + fault Δ bar
08	History → Export All Summaries CSV → verify download

cd windows-memory-analyzer → **streamlit run app.py**

Testing, results & limitations

Verified on Windows 11 Home Build 26200 · Python 3.12.1 · 15.4 GiB RAM

97 / 97

AUTOMATED TESTS PASSED

test_metrics.py (44 tests)

bytes_to_mb/gb conversions, format helpers · counter_delta with DWORD wrap-around · counter_rate with zero/None guards · fault_deltas_from_samples: empty, single, None

test_workloads.py (15 tests)

validate_config — all 5 workload types valid · boundaries: max alloc, min duration, min interval · reject zero/negative alloc, unknown workload, non-numeric

test_storage.py (22 tests)

create/get/list/delete experiment · sample add, count, cascade delete · update_summary, update_status, multi-exp isolation · samples_to_csv / summaries_to_csv output

test_comparison.py (16 tests)

Requires ≥ 2 experiments · compatibility warnings for alloc/duration mismatch · row/chart structure, fault_rate computed from delta

VERIFIED RUNTIME RESULTS

Windows API smoke test (live)

Total RAM	**15 775 MiB (15.4 GiB)**
Available RAM	**~7 048 MiB at idle**
Page size	**4 096 B (confirmed)**
Process count	**~280 processes**
System cache	**~7 430 MiB**
Commit total	**~13 019 MiB**

Self-process (python.exe) test

Working Set	**22.44 MiB**
PageFaultCount	**20 408**
Private Bytes	**14.78 MiB**
Error field	**None**

App health check

HTTP GET /healthz	**200 OK**
Streamlit localhost:8501	**Running**

KNOWN LIMITATIONS

- PageFaultCount does NOT split soft vs hard faults — Windows only via heavier Perf Counters API
- Counter is 32-bit DWORD — wraps at 2^{32} faults; handled in code, unlikely in short runs
- Dashboard auto-refresh sleeps in main thread — avoid during live experiments
- First subprocess spawn ~1 s overhead on Windows (spawn start method, no fork)
- Protected system processes return access-denied — handled gracefully with error field
- Polling-based UI refresh — charts update only on each st.rerun() call
- Allocation capped at min(256 MiB, 10% RAM) — intentionally conservative

CONCLUSION & FUTURE SCOPE

What was achieved — and what can be extended next

WHAT WAS ACHIEVED

- Complete, running Windows memory analysis app — not a prototype
- 3 native Windows APIs integrated via ctypes without admin privileges
- 5 controlled workloads (A–E) in isolated subprocesses — UI stays responsive
- Per-interval capture: PageFaultCount · Working Set · Private Bytes · Available RAM
- Page-fault Δ and fault rate with DWORD wrap-around handling
- 7 Plotly visualisations: gauges · trends · dual-axis · bar · overlay
- SQLite persistence (WAL) + CSV export for summaries and raw samples
- Comparison page with compatibility warnings and time-series overlays
- 97 unit tests — 100% pass — metrics, validation, storage, comparison
- Accurate documentation: PageFaultCount ≠ disk reads, throughout UI and tests

FUTURE SCOPE (NOT IMPLEMENTED)

Hard Fault Separation
Integrate Windows Performance Counters API to separately count hard vs soft faults — the key limitation

ETW Kernel Tracing
Event Tracing for Windows: capture memory events at kernel resolution, sub-millisecond granularity

Working-Set Breakdown
QueryWorkingSet / NtQueryVirtualMemory to categorise pages by type (private, shared, mapped file)

Multi-Process Experiments
Concurrent workloads across multiple PIDs to study inter-process memory pressure and eviction

Statistical Baselines
Record 'golden runs'; flag statistically significant deviations across repeated experiments

Linux Parity Module
/proc/self/status + perf_event_open() collectors for full cross-OS comparison (complete course scope)

THANK YOU

QUESTIONS WELCOME

Source: windows-memory-analyzer/
streamlit run app.py